

PHASE 3 OF 8

System Architecture & Core API

Milestones 1 and 2: model selection, configuration, the FastAPI application, request validation and the /generate route.

- Milestone 1
- Milestone 2
- FastAPI
- Schema

Milestone 1 — Model Selection & System Architecture

The current implementation uses **Gemini** through the **google-genai** SDK. The configured model is **gemini-3.5-flash-lite**. The architecture separates Streamlit, FastAPI, Gemini generation and export services, so any one of them can be changed without rewriting the others.

Activity 1.1: Configuration

Environment configuration example (`.env.example`):

```
GEMINI_API_KEY=your_api_key_here
GEMINI_MODEL=gemini-3.5-flash-lite
BACKEND_URL=http://127.0.0.1:8000
CORS_ORIGINS=http://localhost:8501,http://127.0.0.1:8501
DEMO_MODE=false
```

Variable	Purpose
GEMINI_API_KEY	Credential used by the backend to call Gemini (kept private).
GEMINI_MODEL	Name of the Gemini model to use for generation.
BACKEND_URL	Address the Streamlit frontend uses to reach FastAPI.
CORS_ORIGINS	Frontend origins the backend will accept requests from.
DEMO_MODE	Demo-mode switch; false means live Gemini generation.

Why keep the model name in configuration?

Because the model is read from `GEMINI_MODEL`, switching to a different Gemini model is a one-line change in `.env` — no code edits and no redeployment of the frontend.

Activity 1.2: FastAPI Application

backend/main.py — core application setup:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routes import router

app = FastAPI(title="LegalEase API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:8501",
                  "http://127.0.0.1:8501"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(router)

@app.get("/")
def root():
    return {"message": "LegalEase API is running"}

@app.get("/health")
def health():
    return {"status": "ok"}
```

This module initializes FastAPI, enables local CORS for the Streamlit frontend, includes the document routes and exposes simple service-status endpoints.

Endpoint	Method	Response	Use
/	GET	{"message": "LegalEase API is running"}	Quick confirmation the API is up
/health	GET	{"status": "ok"}	Health check for monitoring or scripts
/generate	POST	{"document": "..."}	Generates the legal draft (defined in routes.py)

About CORS

Browsers block a page on one origin from calling an API on another unless the API allows it. Here the backend allows only the two local Streamlit addresses, which is the safest setting for local development.

Milestone 2 — Core Functionalities Development

Activity 2.1: Request Validation

backend/schemas.py — request model:

```
from pydantic import BaseModel

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    effective_date: str
    jurisdiction: str
    terms: str
    additional_instructions: str = ""
```

The request schema gives the backend a consistent structure for the information collected by the frontend. Five fields are required; additional_instructions is optional and defaults to an empty string.

Activity 2.2: Document Generation Route

backend/routes.py — /generate endpoint (representative excerpt):

```
router = APIRouter()
generator = GeminiDocumentGenerator()

@router.post("/generate")
def generate_document(request: DocumentRequest):
    try:
        document = generator.generate_document(
            document_type=request.document_type,
            parties=request.parties,
            effective_date=request.effective_date,
            jurisdiction=request.jurisdiction,
            terms=request.terms,
            additional_instructions=request.additional_instructions,
        )
        return {"document": document}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))
```

The route receives the structured request, calls the Gemini generator and returns the generated document to Streamlit.

Status	When it occurs	Body
200	Draft generated successfully	{"document": "..."}
422	A required field is missing or has the wrong type (Pydantic validation)	FastAPI validation details
502	Gemini call failed (network, quota, model or key problem)	{"detail": ""}